RAPPORT TECHNIQUE — CARTOGRAPHIE & ÉTAT DES LIEUX

Échange d’informations entre agents

Les quatre couches de coordination d’EFE, leur maturité **réelle**, et le travail Couche 3 M1 : servir de **vrais** participants efe-core sur gRPC, sous garde *fail-closed* et sans sur-vente.

Programme	EFE → Kovant · dossier de preuve (« dot »)
Périmètre	monorepo Rust efe · HEAD 665ead0 (+ working tree)
Date	14 juillet 2026
Auteur	R. Pekar — avec Claude Code (Opus 4.8)
Méthode	cartographie multi-agents du code + revue d’honnêteté ; citations fichier:ligne
Règle cardinale	NE JAMAIS SUR-VENDRE — chaque revendication est étiquetée par sa maturité vérifiée

Avertissement d’honnêteté (à lire d’abord)

Ce rapport distingue systématiquement **ce qui est servi par défaut**, **ce qui est réel et testé mais désactivé par défaut**, et **ce qui est un stub / une réponse en dur**. Le système échange de l’information entre agents de **quatre** façons authentiquement différentes, réparties sur **quatre binaires indépendants** — et **une seule** est active par défaut. Aucune agrégation robuste (vote pondéré, médiane, rejet d’aberrants, tolérance byzantine) n’est servie où que ce soit ; aucune non-expansivité n’est *prouvée* (seulement *falsifiable* par une sonde qui mesure une borne inférieure). Lire la maturité **par chemin**, jamais la capacité agrégée.

1 Résumé exécutif

EFE fait « communiquer des agents » de quatre manières réellement distinctes ; une seule est active par défaut. Ce rapport les cartographie honnêtement, puis détaille le seul chantier que **nous** avons mené sur ce sujet durant cet arc — la Couche 3 M1, qui a rendu *servable* un vrai participant efe-core comme opérateur d’échange du méta-essaim, chaque pas verrouillé *fail-closed* et chaque sur-vente retirée.

(A) Couche 1 — le blackboard latent de l’essaim. À l’intérieur d’un essaim, les « nano-agents » sont des lignes d’un tenseur latent partagé (bf16) ; ils n’échangent que leur vecteur, à travers une fusion par attention-softmax sur CPU qui les tire vers un centroïde (moyenne de Fréchet). C’est réel, déterministe, testé unitairement, et **servi par défaut** dans efe-server (`EFE_BACKEND=mvp`). C’est la couche la plus mûre — mais c’est un produit matriciel en champ moyen sur une ardoise partagée, **pas** des agents autonomes qui négocient des messages (un agent est une ligne de tenseur ; sa voix est un vecteur ; l’échange est un *matmul*).

(B) Couche 2 — le panel frontière multi-LLM. Un panel Claude / GPT / Gemini éclate réellement un prompt vers N modèles et échange du texte — mais seule la mission *debate* passe de vrais arguments ; *vote / relay / specialist / cascade* fusionnent par pluralité / Borda / Condorcet, sous un reçu de provenance re-vérifiable. Cette couche est désormais **servie mais désactivée par défaut** : uniquement dans le binaire séparé efe-swarm-gateway, derrière une feature de compilation `frontier-llm` (défaut off), un drapeau runtime et une triade de démarrage *fail-closed*. Le serveur produit efe-server n’en dépend pas du tout.

(C) Couche 3 — le méta-essaim couplé sur gRPC. Le contrôleur couple des participants en faisant circuler un vecteur de méta-consensus comme « tirage » (*pull*) à chaque pas. Le transport, le mTLS, l’itération couplée, les plafonds d’annulation/deadline et les gardes *fail-closed* sont réels et servis — mais le participant **par défaut** est un mock synthétique $\Phi(x)=\kappa\cdot x$, agrégé par une simple moyenne arithmétique (`UniformMetaOp`) **sans aucune robustesse**. Le vrai participant efe-core, la sonde de Lipschitz empirique (H1) et les bornes **mesurées** (non prouvées : $\hat{L}\approx 0.37$ fixe / 0.69 dynamique, réfutant le certificat analytique $\beta_1\approx 0.17$) vivent tous derrière la feature `core-integration`, désactivée par défaut. **C’est ici que se situe tout notre travail.**

(D) Couche 4 — le blackboard agentique N21. Un quatrième substrat, le plus *littéralement* « passage de messages » : un blackboard (magasin clé-pointeur-JSON versionné) + un *EventBus* (`tokio::broadcast`) + un orchestrateur de DAG + un pont agentique qui appelle le **vrai** Claude (Anthropic Messages API, protocole *tool_use* à deux tours), servi par son propre binaire efe-gateway. Mais **par défaut**, sans clé API, l’endpoint renvoie une réponse en dur ; l’*EventBus* n’a aucun abonné au runtime ; et le DAG servi n’injecte jamais le texte d’entrée, si bien que le *ContextBundle* produit est essentiellement vide.

> **La phrase qu’il faut retenir**
>
> La **seule** forme d’échange inter-agents active par défaut est la fusion latente par attention de la Couche 1, dans efe-server. Toute histoire de « coordination d’agents » inter-modèles ou inter-processus est réelle et testée, mais **désactivée par défaut** ou **mock par défaut**. Aucune agrégation robuste n’est servie (moyenne arithmétique seule) ; aucune non-expansivité n’est prouvée (seulement falsifiable) ; et les quatre binaires sont **indépendants, sans chemin de données partagé à l’exécution**.

2 Cadre : qu’est-ce qu’« échanger de l’information entre agents » ?

Le mot « agent » recouvre ici quatre échelles très différentes, et la nature de « l’information échangée » change radicalement de l’une à l’autre. Confondre ces échelles est la première source de sur-vente ; les séparer est la condition d’un discours honnête.

- **Nano-agent** (Couche 1) — une *ligne* d’un tenseur latent. Son « message » est un vecteur de $\mathbb{R}^{16}$; « échanger » = une attention softmax sur toute la population, puis un repli vers le centroïde.
- **Modèle frontière** (Couche 2) — un LLM (Claude / GPT / Gemini). Son message est du *texte* ; « échanger » = éclater un prompt, collecter les réponses, et les fusionner par vote/Borda/Condorcet (ou, en *debate*, réinjecter l’argument adverse au tour suivant).
- **Participant du méta-essaim** (Couche 3) — un essaim entier vu comme un opérateur Φ. Son message est un *vecteur d’état* transporté par gRPC ; « échanger » = un point fixe couplé où le contrôleur renvoie le méta-consensus comme tirage.
- **SwarmAgent du DAG** (Couche 4) — un nœud d’un graphe orchestré. Son message est une *cellule JSON* écrite sur un blackboard partagé (+ un événement publié sur un bus) ; « échanger » = lire/écrire des pointeurs JSON versionnés.

Un fait structurant, à énoncer d’emblée : ces quatre couches vivent dans **quatre binaires distincts** (efe-server, efe-swarm-gateway, efe-meta-swarm-server, efe-gateway) qui **ne partagent aucun chemin de données à l’exécution**. Le seul lien inter-binaire trouvé dans le code est un *type* de métriques (`SubstrateMetrics`) réutilisé par efe-swarm-gateway pour son endpoint `/metrics` — pas une instance de blackboard, pas un bus, pas un essaim partagé. Toute prose suggérant que « les binaires se coordonnent via un blackboard commun » serait fausse.

3 Panorama des quatre couches

Le tableau suivant est le grand livre de maturité, à lire avant toute affirmation de capacité. La colonne « statut par défaut » est décisive : elle dit ce qui s’exécute réellement dans un binaire compilé et lancé sans drapeau ni clé.

Couche	« Agent » =	Ce qui s’échange	Binaire / transport	Statut par défaut
1 · Blackboard latent	ligne de tenseur	vecteur latent $\mathbb{R}^{16}$ via attention-softmax → centroïde	efe-server (in-proc)	**SERVI**
2 · Panel frontière	LLM (Claude /GPT/Gemini)	texte + reçu de provenance (vote/Borda/Condorcet, debate)	efe-swarm-gateway (HTTP)	**OFF** (feature+flag)
3 · Méta-essaim gRPC	essaim = opérateur Φ	vecteur d’état (pull couplé) sur mTLS	efe-meta-swarm-server (gRPC)	**MOCK** κ·x
4 · Blackboard agentique	nœud de DAG	cellule JSON versionnée + événement ; tool_use Claude	efe-gateway (HTTP)	**CANNED** sans clé

« SERVI » = actif dans un binaire par défaut. « OFF » = réel + testé, mais derrière une feature de compilation et un drapeau runtime. « MOCK » = le transport est servi, mais le calcul par défaut est un substitut synthétique. « CANNED » = l’endpoint répond en dur tant qu’aucune clé API n’est fournie.

Deux corollaires d’honnêteté découlent du tableau. Premièrement, la maturité **décroît** à mesure que l’« agent » devient plus autonome : la couche la plus mûre (1) est aussi la moins « agentique » (un *matmul*), et la couche la plus littéralement agentique (4) est celle dont la réponse par défaut est en

dur. Deuxièmement, **aucune** couche ne sert aujourd'hui d'agrégation robuste : partout où des voix d'agents sont fusionnées sur le chemin servi, c'est une moyenne (arithmétique en Couche 3, pluralité/Borda en Couche 2) sans rejet d'aberrants ni pondération apprise.

4 Couche 1 — le blackboard latent de l'essaim (nano-agents)

Statut : **PROD · servi** · la couche la plus mûre.

Le substrat est l'**Epistemic Latent Space** (`efe-core/src/els/mod.rs`) : quatre tenseurs SoA adossés à une arène — le latent `[N,D]` en bf16 (une ligne $h_i \in \mathbb{R}^D$ par nano-agent, le médium d'échange), un masque d'activité, les ancres, et le *problem embedding*. Un nano-agent n'émet ni jeton ni message : tout son état — et toute son « parole » — est sa ligne latente. À la genèse (`DefaultSpawner`), chaque ligne reçoit h_i = prior[template$_i$] + init_sigma·N(0,1;seed) : un prior de gabarit (sa « persona ») plus un bruit déterministe par-spec. C'est là que l'opinion initiale de chaque agent entre dans le latent partagé.

L'échange proprement dit est la **fusion** (`MvpCpuFusion, fusion/mvp_cpu.rs`). À chaque ronde : on calcule les scores d'attention $s_{ij}=(h_i \cdot h_j)/\sqrt{D}/T$ sur toute la population, un softmax par ligne (stabilisé par décalage-max), puis l'agrégat $new[i]=\Sigma_j\ softmax_{ij} \cdot old[j]$ dans un scratch double-tamponné (échange de pointeurs → coût mémoire O(1) indépendant du nombre de rondes) ; enfin un mélange-β vers le *problem embedding*, une projection sur le cône d'alignement, et un clip. Cette attention dense N×N **est** l'échange d'information entre nano-agents : chaque agent lit la ligne de tous les autres et réécrit la sienne comme un mélange pondéré par consensus. La boucle est pilotée par le `SwarmController` jusqu'à ce que le détecteur de consensus déclenche *Converged* (avec un plancher `min_rounds=4` en Mode-1). La réponse collective de l'essaim est un repli sur le centroïde (`WeightedMeanExtractor`, moyenne arithmétique uniforme sur les lignes actives), extrait par le `FusionAdapter` et publié pour le dissolveur.

Ce chemin est **réellement servi** : efe-server démarre en Mode-1 (`EFE_BACKEND=mvp` par défaut), construit un vrai `SwarmController` et exécute la chaîne genèse → fusion → centroïde. C'est, sans exagération, la partie la plus solide de toute l'histoire inter-agents ; la sous-estimer serait aussi malhonnête que de gonfler les stubs.

> **Ce que la Couche 1 n'est PAS (garde-fous contre la sur-vente)**
>
> • **Pas de négociation d'agents** : c'est une fusion en champ moyen (un matmul), pas des agents qui débattent. • **Pas d'auto-guérison** : la reprise après effondrement de mode (`respawn_diversify`, `inject_noise`) est un *no-op* journalisé — honnête seulement parce que le câblage Mode-1 désactive la reprise. • **Pas de Langevin/SDE** : la ronde servie est attention + mélange-β + projection + clip, sans bruit gaussien par ronde (le mot « Langevin » n'apparaît que dans une docstring de trait). • **CPU seul** : les noyaux CUDA sont des notes « Sprint 3+/7 » — aucune accélération GPU n'est servie. • **Moyenne uniforme** : la pondération robuste aux aberrants (`DistanceInverse`) est « Sprint 8+ », donc quelques lignes aberrantes peuvent tirer le centroïde.

Enfin, une note d'état : le correctif « Φ pur par construction » (porter le centroïde sur l'ELS via `els.dissolution_centroid` / `ElsCarriedDissolver`, pour rendre Φ pur sous `phi_once` concurrent) est **non commité** dans le working tree (diff : els +49, controller +23, mode1 +187, test +94) et n'affecte que le contrôleur latent du méta-essaim, jamais le chemin servi (byte-identique). Un clone propre à HEAD ne l'aurait pas. À reporter comme *en cours*, pas comme une garantie servie de la Couche 1.

5 Couche 2 — le panel frontière multi-LLM (Claude / GPT / Gemini)

Statut : **réel · off par défaut** · servi *uniquement* dans efe-swarm-gateway, feature+drapeau, absent du serveur produit.

Le transport physique est le **MultiLlmBridge** (`efe-coordinator/src/llm/bridge.rs`) : un éclatement parallèle vers N fournisseurs (`FuturesUnordered`), collectant une réponse par fournisseur. Les backends sont de **vrais** clients HTTP reqwest vers `api.anthropic.com/v1/messages`, `api.openai.com/v1/chat/completions` et `generativelanguage.googleapis.com` (clés via `from_env`). Cinq missions incarnent les topologies d’échange : *vote* (éclatement N-voies puis agrégation), *debate* (LLMs pro/contra en alternance, où le prompt du tour N encapsule l’argument adverse du tour N−1 — le seul vrai échange de transcription), *relay* (chaîne plan→code→synthèse), *specialist* (décomposition→N experts→agrégation), *cascade* (rédacteur→vérificateur→validateur).

La fusion des réponses passe par un reformulateur (chaque sortie brute devient une *CanonicalClaim*) puis un décompte par pluralité, Borda ou Condorcet. Un fait d’honnêteté : le « classement inter-modèles » de Borda est une **heuristique** (chaque LLM se classe lui-même premier, puis ordonne les autres par similarité de Jaccard — commentaire « Heuristique simple ») : les modèles ne se classent pas réellement les uns les autres ; seul *debate* réalise un vrai passage d’arguments. La pondération est uniforme — **pas** de vote pondéré ni de rejet d’aberrants.

Depuis l’arc COUCHE2 (commits f71f5b8 / c7ed62d / f8e70a5), cette couche a une **surface servie** : `POST /v1/llm/{vote,relay,debate,specialist,cascade}` et `GET /v1/llm/audit/:id` dans efe-swarm-gateway, chaque mission sous une portée de provenance qui scelle un **reçu re-vérifiable**. Le graphe causal est construit depuis la topologie **déclarée** (pas les enregistrements observés), si bien qu’un appel d’egress manquant devient un nœud non-lié → `verified=false` : cela ferme le trou « reçu vide re-prouvé totalement valide ». Une porte de souveraineté *fail-closed* (`egress_gate`) refuse (403) tout LLM hors liste ou toute politique de tenant absente **avant** le moindre appel.

> **Le double verrou — et pourquoi « en production » serait faux**
>
> « Servi » ici est doublement conditionné et absent du serveur produit : (1) feature de compilation `frontier-llm` dont le défaut est vide (off) ; (2) runtime `EFE_LLM_FRONTIER_ENABLED=true` exact, plus une triade de démarrage *fail-closed* (clé HMAC ≥ 32 o, clé publique OIDC aud=efe-llm, magasin Postgres durable) sans quoi le boot échoue. Et efe-server (le serveur produit) a **zéro** dépendance sur toute cette pile. Écrire « la plateforme coordonne Claude/Gemini/GPT en production » serait faux pour la build par défaut.

- **Résidu de silence** : la mission *vote* appelle encore `call_many_ok` (`vote.rs:57`) — un fournisseur en échec au niveau du bridge est retiré de la logique de vote (le décorateur de provenance enregistre tout de même un nœud *Errored* pour les appels ayant atteint le fournisseur, donc le reçu n’est pas aveugle).
- **Intégrité ≠ correction** : le reçu prouve *ce qui s’est passé* (quels modèles, quels hachages, quel vote), pas que la réponse est juste. La distinction est estampillée dans le code (`HeldStrengthView`, rung R1..R4).
- **Garde d’egress dupliquée** : le classificateur de localité (`is_local_endpoint/host_of`) existe en deux copies synchronisées à la main (provenance-capture vs efe-swarm-sdk/backend) ; un dérapage SSRF historique est noté et re-synchronisé en commentaire — à durcir ensemble (risque de maintenance, pas bug actuel).

- **efe-llm-router** : explicitement **rejeté** comme porteur de coordination (audit Postgres non re-prouvable hors-ligne, temperature=0 tuant la diversité) ; gelé, non servi.

6 Couche 3 — le méta-essaim couplé sur gRPC (notre chantier)

Statut : transport **PROD · servi** · participant réel **réel · off par défaut** · défaut = **mock uniquement** (κ·x).

C'est ici que se concentre **tout notre travail**. Le contrat de fil (`protos/efe/v1/meta_swarm/`) définit l'échange : côté contrôleur `Iterate/StepOnce/Reset/ReloadConfig/Health/Info`, côté participant `Step/State/Capabilities/Health`. L'état sur le fil est un `TensorF32` dense ; `StepRequest.pulled_from_meta` est le vecteur échangé à chaque pas. Le moteur est une **itération couplée** (`CoupledIteration`) : un point fixe où le contrôleur fusionne les votes des participants en un méta-consensus puis le redistribue comme tirage, selon $x_{t+1} = \mathrm{mix}(x_t, \Phi(x_t), \mathrm{pull} ; \alpha, \gamma)$. Le transport (`RemoteParticipant` → `Step` RPC sur mTLS), l'annulation, les plafonds de deadline : tout cela est réel et servi.

Mais le participant **servi par défaut** est le `MockParticipant` — un $\Phi(x)=\kappa\cdot x$ synthétique, non-expansif par construction — et l'agrégateur est `UniformMetaOp` (moyenne arithmétique, borne de Lipschitz exactement 1). C'est le **seul** MetaOperator jamais instancié où que ce soit (défaut du builder *et* câblage serveur, `iterate.rs:701`). L'alternative `AttentionMetaOp` (softmax) est un stub explicitement « non validé contre le théorème de convergence », derrière une feature off, dont la borne $1+2\|q\|/\tau$ peut **dépasser 1** (expansif). Il n'y a **ni** rejet d'aberrants, **ni** médiane/trimmed, **ni** pondération apprise nulle part.

6.1 · Le vrai participant efe-core (réel, mais opt-in)

Le participant réel mappe l'état méta x → un *problem embedding* efe-core, exécute une ronde de fusion numérique Mode-1 (`build_mvp_controller` / `build_fixed_topology_controller`), et renvoie Φ(x). Ce sont de **vrais essaims latents efe-core** — **pas** des SLM : ni poids de modèle, ni tokenizer sur le chemin du pas ; l'échange y est un vecteur-centroïde de $\mathbb{R}^{16}$, pas une inférence de modèle. La feature `core-integration` est **désactivée par défaut** dans les deux crates ; sur une build par défaut, une ligne de manifeste efe-core **échoue à la construction** (« built without core-integration ») — fail-closed, jamais un repli silencieux sur le mock.

6.2 · L'hypothèse H1 et sa sonde — un falsificateur, pas une preuve

La convergence (Krasnoselskii–Mann) repose sur l'hypothèse H1 : Φ non-expansif ($L_\Phi \le 1$). La vraie menace est la **discontinuité** du routage génésique (la classe de x → un nombre **entier** d'agents → une topologie en escalier → Φ discontinu aux frontières). La sonde de Lipschitz empirique (`lipschitz.rs`) mesure une borne **inférieure** $\hat{L}$ par échantillonnage adverse en boîte noire avec bisection vers les discontinuités. C'est un **falsificateur** : il peut **rejeter** H1 ($\hat{L} \ge 1-\text{marge} \Rightarrow$ refus), jamais la **certifier**. Chaque étiquette dit « non-expansivité property-testée, NON certifiée ».

> **Le certificat analytique $\beta_1 \approx 0.17$ est RÉFUTÉ par la mesure**
> La borne intra-cellule en forme fermée $L_\Phi \le \beta_1 \approx 0.17$ n'est **émise nulle part** comme certificat. La sonde M1.c mesure $\hat{L} \approx$ **0.366** sur l'opérateur fixe et $\hat{L} \approx$ **0.691** sur le dynamique — tous deux < 1 (H1 tient, property-testé) mais tous deux **très au-dessus** de 0.17 : preuve directe d'une sensibilité continue à x au-delà de *beta_mix*. La seule preuve H1 honnête est la borne **mesurée**.

Un point d'honnêteté que nous avons **nous-mêmes** corrigé cet arc (finding #2) : l'écart 0.366 vs 0.691 est **confondu**, pas causal. `GenesisEngine::spawn` ignore `spec.population` et le chemin dynamique utilise `HeuristicEstimator` (`min_agents=128`) : l'opérateur fixe exécute 4 agents *na-sem*

purs quand le dynamique en fait éclore ≥128 mixtes. La taille de population (~32×) **et** le mix changent en même temps que la fixité du routage ; la revendication « fixer la topologie divise la sensibilité par ~1.9 » a été **retirée**. Ce qui subsiste (énoncé mono-opérateur, non comparatif) : l'opérateur fixe seul mesure 0.366 ≫ 0.17, ce qui réfute le certificat.

6.3 · Les gardes fail-closed réellement câblées

- **h1_boot_gate** (`participant/local.rs:343`) — exécute la sonde **avant** que le participant ne lie son port ; $\hat{L} \geq 1-\text{marge}$ ou non concluant ⇒ refus de servir. C'est la seule vraie garde de non-expansivité de toute la couche (au boot, pour les moteurs efe-core).
- **require_convergence** (`iterate.rs:211`) — si activé, un Iterate non convergé renvoie gRPC `ABORTED` au lieu d'un « consensus » non fiable. Garde au niveau réponse (défaut off ; StepOnce exempté par conception — un pas unique n'a pas de verdict de convergence).
- **config_hash sur le fil** (`messages.proto`, champ 8) — les lignes efe-core portent un bloc `controller` dont le SHA-256 canonique est annoncé ; présence ⟺ vrai moteur ; l'absence **doit** se lire « non vérifié / mock » (un vieux binaire élague le champ inconnu → fail-closed).
- **topologie fixe** (Palier-2, `genesis/fixed.rs`) — rend la population x-indépendante, donc Φ continu par construction (élimine la discontinuité du routage, l'unique tueur de H1). Topologie par défaut des lignes efe-core ; le *dynamic* est un opt-out explicite, visible sur le fil, journalisé au boot.

> **Ce qui reste stub en Couche 3 (à ne pas gonfler)**
> Les **certificats de fil** sont des stubs : `RemoteParticipant::verify_non_expansive` renvoie toujours `Stub`, donc la pré-vérification H1 par Iterate est un *no-op* sur le chemin servi (mode indulgent — le mode strict rejetterait tout Stub et briquerait le service). H1 n'est **ni échangé ni vérifié à l'exécution** entre agents ; il n'est contrôlé qu'au **boot** du participant, et seulement pour les moteurs efe-core (eux-mêmes off par défaut). Les certificats signés sur le fil (Ed25519 `LocalSignature`) sont le jalon M4, **non construit**. Les *lifts/pulls* servis sont l'identité (pas de projection inter-espaces apprise). Seule la géométrie `BoxSafety` est câblée (Poly/Hull rejetées au build).

7 Couche 4 — le blackboard agentique N21 (efe-gateway)

Statut : substrat **PROD · servi** · pont Claude **réel · off par défaut** · réponse **mock uniquement** (en dur) sans clé.

C'est la couche la plus **littéralement** « passage de messages », servie par son propre binaire axum efe-gateway (distinct des trois autres). Le blackboard (`efe-blackboard/src/blackboard.rs`) est un `DashMap` clé-pointeur-JSON : chaque cellule porte sa valeur, une **version monotone par pointeur**, l'agent auteur et l'horodatage. L'écriture conditionnelle `write_if` est un vrai compare-and-swap sous un seul verrou de shard (correctif d'atomicité post-audit). S'y ajoutent des plafonds d'octets globaux et par-espace, un anneau borné des écritures récentes, et des seams d'observabilité. L'*EventBus* (`eventbus.rs`) est un `tokio::broadcast` borné (capacité 256) publiant des `SwarmEvent`.

L'orchestrateur (`efe-swarm-orchestrator/src/runner.rs`) alloue **un** blackboard et **un** bus neufs par exécution, puis déroule les couches du DAG : chaque agent non-sauté d'une couche est lancé en parallèle (`tokio::spawn`) avec un timeout par-agent dans un timeout global, arêtes conditionnelles et propagation de saut. Le pont agentique (`efe-agentic-bridge/src/claude.rs`) appelle le **vrai** Claude : client reqwest HTTPS vers `api.anthropic.com/v1/messages`, protocole *tool_use* à deux tours (un outil fixe `swarm_context`, le *ContextBundle* réinjecté comme `tool_result`), streaming SSE, retry à back-off, et provenance N17 fail-closed. C'est un client Anthropic substantiel et réel.

Pourquoi « Blackboard essaim = PROD » n’est vrai qu’à moitié

Trois réserves décisives, toutes vérifiées dans le code : (1) **Réponse en dur par défaut** — sans `ANTHROPIC_API_KEY`, le gateway sélectionne `StubAgenticBridge`, et `/v1/agentic/call` renvoie « [stub bridge] would have called Claude... ». Le vrai appel Claude est réel-mais-off-par-défaut. (2) **ContextBundle essentiellement vide** — le DAG servi n’ensemence jamais `/input/text` (`seed_input_text` n’est appelé qu’en test), et l’agrégateur est un `NoOpAggregator` : les agents d’analyse lisent un texte vide et le bundle ne récapitule que des identifiants d’agents et un coût. (3) **EventBus sans abonné** au runtime — chaque publication retombe en écriture blackboard. La machinerie de concurrence tourne vraiment ; la charge utile analytique, elle, est quasi nulle sur le chemin servi.

À créditer néanmoins : le substrat de concurrence (versioning par-champ, CAS atomique, plafonds, anneau borné) est réel et audité ; l’orchestrateur (ordonnancement parallèle par couches, timeouts, isolation best-effort) est réel ; le client Claude est un vrai client Anthropic (deux tours, streaming, retry, provenance) ; le gateway applique une vraie authentification JWT/tenant. La **capacité** « bus pub/sub pour événements inter-agents » est exacte ; ce qui manque, c’est un **abonné vivant** et un ensemencement d’entrée sur le chemin servi.

8 Ce que nous avons construit — le fil chronologique (M1.a → 665ead0)

Notre travail sur l’échange inter-agents est l’arc Couche 3 M1 : ouvrir, durcir, puis rendre *servable* (en opt-in) un vrai participant efe-core comme opérateur Φ du méta-essaim. Sept commits, ~22 h, du 13 au 14 juillet 2026. Chaque jalon a été verrouillé fail-closed ; chaque sur-vente, retirée.

Commit	Jalon	Livré (et le piège)	Statut
8fcb3dc	M1.a — couture du participant	Ouvre la couture : le pas méta appelle un vrai `SwarmController` via `phi_once` ; `step_with_pull` → mix couplé. *Piège* : Mode-1 a min_rounds=4/hystérésis → il fallait un détecteur « converge-now ».	réel, testé
720d07c	M1.b — Φ déterministe	`init_sigma=0` sur le contrôleur latent → Φ bit-reproductible. *Piège* : la valeur réelle est l’invariance au seed (le bruit de genèse dérive de l’id de spec, pas du SwarmId frais).	réel, testé
a8f1798	M1.c — sonde de Lipschitz	Ranime la garde H1 : estimateur adverse (bisection-discontinuités), certificat empirique, garde stricte. *Piège* : un bug *fail-open* (`.ok()` avalait les explosions de Φ) → corrigé via un `PhiOutcome` à 3 états.	réel, off
c53a7f6 + d3ec501	M1.d — participants SERVIS	Câble le pont efe-core dans la couche gRPC (bloc moteur, config_hash, boot-gate, require_convergence). *Piège* : un .gitignore trop générique avalait des dossiers SOURCE → le clone ne compilait pas.	réel, off
b9a8fce	Palier-2 — topologie fixe	Élimine la discontinuité du routage (Φ continu par construction). **La mesure qui tranche** : L̂ 0.366 (fixe) vs 0.691 (dynamique) — le certificat analytique $\beta_1 \approx 0.17$ est réfuté.	réel, off
665ead0	Solde de la dette de revue	Ferme la revue adverse M1.d (10 findings). #1 = un vrai bug de solidité (bypass d’identité-moteur au reload) ; #2/#6/#8 = retraits de sur-vente (le ratio ~1.9 confondu, docstrings trop affirmatives).	corrigé

Contexte de la couche voisine : l’arc COUCHE2 (décision + surface servie) est antérieur et distinct — il a doté la Couche 2 d’un reçu re-vérifiable dans efe-swarm-gateway, avec le modèle de confiance honnête « R4 = infalsifiable / vérifiable par le détenteur de clé, PAS tiers-de-confiance ».

Deux méta-faits d’honnêteté sur cette chronologie. D’abord, **le jalon servi a précédé sa revue** : M1.d a été commité sous contrainte de crédits, sa revue adverse initialement reportée, puis soldée ~12 h plus tard (665ead0) — dont un vrai bug de solidité. Ensuite, le correctif de .gitignore (d3ec501) signifie que le dépôt poussé **ne compilait pas** depuis un clone neuf avant le milieu de l’arc (le module manifeste et efe-telemetry/logs étaient non suivis).

9 Le grand livre d’honnêteté

Cette section est la colonne vertébrale de crédibilité du rapport : pour chaque revendication *tentante*, la *réalité* vérifiée dans le code. C’est le filtre à passer avant toute présentation à Kovant.

Revendication tentante	Réalité vérifiée dans le code
« La plateforme coordonne Claude/Gemini/GPT en production. »	**Faux pour la build par défaut.** Servi uniquement dans efe-swarm-gateway, feature `frontier-llm` (off) + drapeau + triade de boot. efe-server n’en dépend pas.
« Le méta-essaim fait tourner de vrais essaims/SLM. »	Par défaut c’est un **mock** Φ=κ·x. Le vrai participant efe-core est off (`core-integration`), et ce sont des **essaims latents numériques**, pas des SLM (ni poids ni tokenizer).

Revendication tentante	Réalité vérifiée dans le code
« Agrégation pondérée / robuste / byzantine entre agents. »	**Aucune.** Seul `UniformMetaOp` (moyenne, Lipschitz=1) est instancié. Un participant adverse déplace le consensus. `AttentionMetaOp` est un stub (peut être expansif).
« Non-expansivité prouvée ($L \le \beta_1 \approx 0.17$). »	**Réfutée comme certificat.** Mesuré $\hat{L} \approx 0.37$ (fixe) / 0.69 (dynamique) $\gg 0.17$. La sonde est un **falsificateur** (borne inférieure), jamais une preuve.
« Fixer la topologie divise la sensibilité par ~1.9. »	**Retiré (confondu).** Fixe = 4 agents, dynamique ≥ 128 (population ~32× + mix changent avec le routage). Seul survit « l'opérateur fixe seul : 0.366 ≫ 0.17 ».
« Certificats de non-expansivité échangés sur le fil. »	**Stub.** `verify_non_expansive` renvoie toujours Stub ; H1 n'est vérifié qu'au **boot**. Les certificats signés (Ed25519) sont le jalon M4, non construit.
« Le blackboard agentique coordonne de vrais agents LLM. »	**Réponse en dur par défaut** (sans clé). De plus le DAG servi n'ensemence pas l'entrée → ContextBundle quasi vide ; l'EventBus n'a aucun abonné runtime.
« L'essaim s'auto-diversifie / récupère de l'effondrement. »	**Stubs no-op.** `respawn_diversify` et `inject_noise` journalisent et renvoient Ok ; le câblage Mode-1 désactive la reprise.
« Les quatre couches se coordonnent via un blackboard commun. »	**Faux.** Quatre binaires indépendants, aucun chemin de données partagé à l'exécution (seul un *type* de métriques est réutilisé).

Il faut aussi **créditer** ce qui est authentiquement solide, car sous-estimer est aussi malhonnête que gonfler : la fusion latente de la Couche 1 (servie, déterministe, testée) ; la sonde de Lipschitz (un vrai falsificateur adverse qui a attrapé son propre bug fail-open en revue) ; les gardes fail-closed (boot H1, require_convergence→ABORTED, asymétrie kappa/seed en Option, garde d'identité-moteur au reload) ; le transport mTLS ; et le reçu de provenance re-vérifiable de la Couche 2. La **machinerie d'honnêteté** est réelle, même là où les garanties sont revues à la baisse.

10 La session courante — solder la dette de revue (665ead0)

Le dernier commit est celui que nous venons de produire : la revue adverse différée de M1.d, ses 10 findings corrigés, re-vérifiés, puis re-passés au crible d'une revue adverse à quatre lentilles (0 finding confirmé).

Le correctif substantiel (#1) — le bypass d'identité-moteur au reload

La vérification de reload ne comparait que les identifiants présents dans **les deux** manifestes : un id **retiré** puis **ré-ajouté** avec un moteur différent, sur deux reloads SIGHUP, échappait au contrôle de dérive (AD5 autorise add/remove). Nous avons ajouté une **garde de lignée** d'identité-moteur (`validate_engine_lineage` + un registre `engine_lineage` amorcé au boot, monotone « première identité gagne »), en factorisant le comparateur partagé `participant_engine_drift`. Note d'honnêteté : le **côté participant** était déjà le rempart sûr (il compare à l'état *courant*, immunisé) ; ceci ne corrige que la divergence du *livre* du contrôleur.

Les retraits de sur-vente et le mordant des tests

- **#2/#6/#8** — retrait du claim causal confondu « ~1.9 » ; docstrings alignées sur la mesure (0.37 > β_1, sensibilité au-delà de *beta_mix*) ; doc `ControllerManifest` corrigée.
- **#3/#4/#5** — chemin REFUSE du boot-gate (marge impossible), garde anti-vacuité de la contraction (`first > 1e-4`), assertion distinctive **non-causale** fixe-vs-dynamique (`|Δ| > 0.1`).
- **#7/#9/#10** — exemption StepOnce documentée + épinglée ; garde de reload participant (step 2bis) testée.

Vérification (avant commit)

Suite par défaut : **20 binaires, 0 échec** (lib 368). Suite feature `core-integration` : **27 binaires, 0 échec** (e2e efe-core sur gRPC 5/5, manifest-engine 7/7, require_convergence 4/4). Re-revue adverse à 4 lentilles (~395 k tokens) : **0 finding confirmé**. Commit `665ead0`, poussé sur `couche2-m0-frontier`.

11 Ce qui reste — feuille de route honnête

Rendre l'échange inter-agents réellement « de production » suppose de refermer, dans l'ordre de valeur, les écarts que ce rapport a nommés sans les masquer :

- **Servir le vrai participant par défaut** — sortir efe-core de `core-integration` off, ce qui exige de câbler les certificats **sur le fil** (jalon M4) pour que le mode strict ne brique pas le service (aujourd'hui la garantie passe par la voie `ABORTED`, pas par des certificats).
- **Une vraie agrégation robuste** — au moins une médiane/trimmed ou un rejet d'aberrants servi, pour que le consensus résiste à un participant adverse (aujourd'hui : moyenne uniforme).
- **Réconcilier l'écart $\beta_1 \approx 0.17$ vs $\hat{L} \approx 0.37$** — attribuer la sensibilité résiduelle par une mesure étage-par-étage (via `phi_once` sur des variantes d'assemblage) ; re-mesurer fixe-vs-dynamique **à population appariée** pour lever le confond.
- **Consolider mode1** — fusionner le contrôleur à topologie fixe (miroir dupliqué côté serveur) dans un `build_mvp_controller_fixed` (dette assumée, dérive à surveiller) ; et lander le correctif « Φ pur » (non commité) proprement.
- **Couche 2** — appliquer complètement `call_many_ok → call_many` (vote), déduplication structurelle de la garde d'egress, et — si souhaité — câbler la surface frontière dans le serveur produit.
- **Couche 4** — ensemencer `/input/text` sur le chemin servi, remplacer le `NoOpAggregator`, et brancher un abonné réel sur l'EventBus, sans quoi le blackboard agentique reste une coquille par défaut.

12 Annexe — index des crates, binaires et jalons

Binaires servant une couche d'échange

Binaire	Couche	Échange servi par défaut	Crates clés
efe-server	1	fusion latente Mode-1 (SERVI)	efe-core, efe-server/mode1.rs
efe-swarm-gateway	2	panel frontière (OFF : feature+flag)	efe-coordinator, efe-multi-llm, provenance-capture
efe-meta-swarm-server	3	méta-essaim gRPC (MOCK κ·x)	efe-meta-swarm, efe-meta-swarm-server
efe-gateway	4	blackboard agentique (CANNED sans clé)	efe-blackboard, efe-swarm-orchestrator, efe-agentic-bridge

Empreinte de tests (comptes statiques par grep — non re-exécutés, sauf mention)

- **Couche 1** (efe-core fusion/genesis/els/dissolution/swarm) : ≈ 427 tests inline (grep).
- **Couche 2** : ≈ 131 fns de test (efe-multi-llm), ≈ 727 attrs (efe-coordinator), ≈ 36 (gateway llm.rs) — grep.
- **Couche 3** : suites M1 **re-exécutées cette session** — défaut 368 lib + 20 bins, feature 27 bins, 0 échec.

- **Couche 4** : ≈ 675 attrs de test sur les 5 crates (grep ; suites `audit_*` / `chaos_*` + Anthropic mocké via wiremock).

> **Note de méthode & réserve finale**
>
> Ce rapport est ancré sur une cartographie multi-agents du code (citations fichier:ligne) et une revue d’honnêteté dédiée. Les comptes de tests des couches 1, 2 et 4 sont des **estimations statiques par grep**, non des exécutions vertes vérifiées ; seules les suites M1 (Couche 3) ont été re-exécutées cette session. Aucun déploiement multi-binaires en production n’est établi ici : l’existence de quatre serveurs ne prouve pas une topologie déployée. En cas de doute, ce document choisit systématiquement la formulation la plus basse — conformément à la règle cardinale.